

Report - Version 2: Data Inspector

Introduction

Version 2 extends the first working version of the Building Agentic AutoML project with a dedicated dataset inspection step.

The agent still receives a dataset and target, detects the task, prepares the data, trains a LightGBM baseline, evaluates it, and returns a structured state. The new capability is that the dataset is profiled before training so the state also contains information about shape, target behavior, feature data types, missing values, and cardinality.

This version focuses on the following flow:

```
Dataset + target
↓
Detect task and features
↓
Inspect dataset
↓
Prepare data and baseline
↓
Evaluate
↓
Return state + inspection
```

The architecture remains intentionally simple: one model, one train-validation split, and no optimization. Version 2 adds observability of the dataset without changing the basic training strategy.

Goal of Version 2

The goal of Version 2 is to make the agent more informed before training while preserving the minimal architecture introduced in Version 1.

The agent should now be able to:

- load a dataset and target;
- distinguish classification from regression;
- identify numerical and categorical features;
- inspect dataset dimensions, target distribution or summary statistics, data types, missing values, and feature cardinality;
- select a suitable model and metric;
- train, evaluate, and return the inspection together with the state.

The main classification example is now the Adult Income dataset, which provides a more realistic tabular structure and includes missing values in several categorical columns.

Architecture

The flow of Version 2 is:

```
Dataset + Target → Task/Features → Data Inspection → Preparation
→ Model/Metric → Evaluation → State
```

The function agent still coordinates the complete flow directly. The new `inspect_dataset` component is called before data preparation, and its result is stored in the final state.

Task Detection

Task detection is unchanged from Version 1. The agent inspects the target column and uses the number of unique values as a simple rule: at most 20 unique values means classification; otherwise the task is treated as regression.

```
def detect_task(df, target):
    n_unique = df[target].nunique()
    if n_unique <= 20:
        return "classification"
    return "regression"
```

For the Adult Income target income, the result is:

```
"classification"
```

Feature Detection

The agent separates numerical and categorical features directly from the dataframe dtypes.

For Adult Income it detects 6 numerical features and 8 categorical features:

```
numerical = ['age', 'fnlwt', 'education_num',
             'capital_gain', 'capital_loss', 'hours_per_week']

categorical = ['workclass', 'education', 'marital_status', 'occupation',
              'relationship', 'race', 'sex', 'native_country']
```

Data Inspection

This is the new component introduced in Version 2. Before training, `inspect_dataset` creates a compact profile of the dataset.

<code>shape</code>	→ rows and columns
<code>target</code>	→ name, dtype, unique values, distribution/summary
<code>dtypes</code>	→ feature data types
<code>missing_values</code>	→ missing count for every column
<code>cardinality</code>	→ number of unique non-null values per feature

For Adult Income, the inspection reports 48,842 rows and 15 columns. The target contains two classes: $\leq 50K$ with 37,155 rows and $> 50K$ with 11,687 rows.

The main missing values detected are `workclass` = 2,799, `occupation` = 2,809, and `native_country` = 857. The inspector also exposes feature cardinality, for example `education` = 16, `occupation` = 14, and `native_country` = 41.

The inspection is descriptive only: it reports the dataset structure but does not yet impute missing values, remove outliers, or alter features.

Data Preparation

As in Version 1, categorical columns are converted to the pandas category dtype and features X are separated from target y . LightGBM then handles the categorical features directly.

Baseline Model and Metric Selection

Model and metric selection remain unchanged. Classification uses LGBMClassifier with ROC AUC, while regression uses LGBMRegressor with RMSE. Both models use random_state=42 and the default LightGBM configuration.

```
classification → LGBMClassifier + ROC AUC
regression     → LGBMRegressor  + RMSE
```

Training and Evaluation

The agent uses a single train-validation split with test_size=0.2 and random_state=42. Classification also uses stratify=y to preserve class proportions.

On Adult Income, the notebook obtains:

```
model  = LGBMClassifier
metric = roc_auc
score  = 0.9311658417981501
```

The score confirms that the original baseline pipeline continues to operate on the more realistic dataset after the inspection step is added.

Agent

In Version 2, the Agent performs the following operations:

1. load dataset and target
2. detect classification or regression
3. identify numerical and categorical features
4. inspect the dataset
5. prepare the data
6. select model and metric
7. train and evaluate the baseline
8. return the final state including inspection

A simplified state now contains:

```
{
  "task": "classification",
  "inspection": {
    "shape": {"rows": 48842, "columns": 15},
    "target": {"name": "income", "unique_values": 2},
    "missing_values": {...},
    "cardinality": {...}
  },
  "model": "LGBMClassifier",
  "metric": "roc_auc",
  "score": 0.9311658417981501
}
```

The state is therefore more informative than in Version 1: it records not only the decisions and evaluation result, but also the dataset profile used before training.

Regression Test

The same agent is tested on simple_regression.csv with target column "target". The agent detects regression automatically and also produces a regression-specific inspection, where the continuous target is summarized with descriptive statistics instead of a class distribution.

```
task    = regression
model   = LGBMRegressor
metric  = rmse
score   = 1.8760451113860066
```

The regression test confirms that the inspection logic adapts to both discrete and continuous targets without changing the agent workflow.

What Version 2 Teaches

Version 2 introduces a fundamental AutoML idea: an agent should understand the structure of the dataset before training a model.

The main idea becomes:

observe → inspect → decide → prepare → train → evaluate → return

The new inspection step makes the agent state more transparent. It exposes dataset dimensions, target behavior, feature types, missing values, and cardinality, which are useful signals for later automated decisions.

This also establishes a clean evolutionary pattern for the project: each version introduces one additional capability while preserving the working baseline whenever possible.

Limitations

Version 2 still has intentional limitations:

- task detection still relies only on the number of unique target values;
- the Data Inspector reports missing values but does not impute them;
- the inspector does not detect outliers, leakage, duplicate rows, skewness, or semantic data types;
- only one model family, LightGBM, is used;
- there is no comparison across multiple algorithms or hyperparameter search;
- evaluation still uses a single train-validation split;
- there is no cross-validation;
- there is no planner or multi-agent architecture yet.

These limitations are deliberate. The purpose of Version 2 is inspection, not automated remediation or optimization.

Conclusion

Version 2 extends the Building Agentic AutoML baseline with a Data Inspector while preserving the simplicity of Version 1.

The agent can now inspect a realistic dataset, expose its structure and quality signals, train the same LightGBM baseline, and return the inspection together with task, feature, model, metric, and score information.

The key lesson is that agentic AutoML becomes more useful when decisions are grounded in an explicit representation of the data. Version 2 provides that first layer of dataset awareness and prepares the project for later versions that can act on the information discovered during inspection.